PROJECT REPORT ON

CALLGUARD

REAL-TIME DEEPFAKE AUDIO DETECTION ON ANDROID

PROJECT REPORT & TECHNICAL GUIDE

Submitted in partial fulfillment of the requirements for the award of **Diploma in Computer Engineering**.

SUBMITTED BY

SHAIK NASZMA	**24004-CS-078**
BAKAM SAI VIGNESH	**24004-CS-124**
NADUKUDOLLA SANJANA	**24004-CS-134**
DASI NAGA CHARITHA	**24004-CS-116**
MANDALA JASHWANTH	**24004-CS-092**
THADIGOPPULA SANJANA	**24004-CS-108**
SAPAVAT PAVANI	**24004-CS-107**
BHUKYA NITHIN KUMAR	**24004-CS-132**
BHUKYA MANASA	**24004-CS-131**

Under the guidance of

V. Veeranna, M.Tech

and Principal

P. Srinivas, M.Tech

DEPARTMENT OF COMPUTER ENGINEERING

GOVERNMENT POLYTECHNIC WARANGAL

Warangal, Telangana • Academic Session 2024–2027

DEPARTMENT OF COMPUTER ENGINEERING

GOVERNMENT POLYTECHNIC WARANGAL

CERTIFICATE

This is to certify that this project report entitled **“CALLGUARD: REAL-TIME DEEPFAKE AUDIO DETECTION ON ANDROID”** is a bonafide record of the project work carried out by the students listed below, in partial fulfillment of the requirements for the award of Diploma in Computer Engineering, during the academic session 2024–2027.

SHAIK NASZMA — 24004-CS-078
BAKAM SAI VIGNESH — 24004-CS-124
NADUKUDOLLA SANJANA — 24004-CS-134
DASI NAGA CHARITHA — 24004-CS-116
MANDALA JASHWANTH — 24004-CS-092
THADIGOPPULA SANJANA — 24004-CS-108
SAPAVAT PAVANI — 24004-CS-107
BHUKYA NITHIN KUMAR — 24004-CS-132
BHUKYA MANASA — 24004-CS-131

The work is submitted for academic review. The system is a research prototype; its results should be treated as an informational aid and not as proof of identity or synthetic speech.

PROJECT GUIDE	**PRINCIPAL**
V. Veeranna, M.Tech	P. Srinivas, M.Tech

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **V. Veeranna, M.Tech**, for his guidance, valuable suggestions, and encouragement throughout the development of this project. His support helped us understand the technical challenges involved in building a practical audio-analysis prototype.

We are thankful to our Principal, **P. Srinivas, M.Tech**, for providing the opportunity and institutional support to undertake this work. We also thank the faculty members of the Department of Computer Engineering for their assistance and feedback.

We acknowledge the public research datasets and open-source technologies used in the CallGuard project, including Android, Jetpack Compose, LiteRT/TensorFlow Lite, PyTorch, and publicly available speech-spoofing corpora. Finally, we thank our families and friends for their encouragement.

PROJECT TEAM

SHAIK NASZMA	**24004-CS-078**
BAKAM SAI VIGNESH	**24004-CS-124**
NADUKUDOLLA SANJANA	**24004-CS-134**
DASI NAGA CHARITHA	**24004-CS-116**
MANDALA JASHWANTH	**24004-CS-092**
THADIGOPPULA SANJANA	**24004-CS-108**
SAPAVAT PAVANI	**24004-CS-107**
BHUKYA NITHIN KUMAR	**24004-CS-132**
BHUKYA MANASA	**24004-CS-131**

ABSTRACT

CallGuard is an Android research prototype for on-device detection of synthetic or deepfake speech. It analyses audio through two supported paths: live microphone input, typically while a call is on speakerphone, and post-call audio-file analysis. Audio is converted to 16 kHz mono and represented as a log-mel spectrogram with 64 mel bands and 198 frames over a two-second window. A lightweight convolutional neural network exported to TensorFlow Lite estimates the probability of synthetic speech. Platt calibration and median smoothing are used to produce three descriptive verdict bands: “Sounds human,” “Not sure,” and “Likely synthetic.”

The prototype is designed to keep processing on the device and does not upload audio. It does not intercept carrier call audio and is not a wiretap, forensic instrument, or proof of impersonation. Reported evaluation illustrates the importance of these limitations: the model performs strongly on the in-distribution ASVspoof 2019 development set (2.5% equal error rate), while cross-dataset performance on newer neural-vocoder audio is substantially weaker (about 40.7% EER). Users should treat every output as a second opinion and verify concerns through another channel.

KEYWORDS

Deepfake audio • Synthetic speech • Android • TensorFlow Lite • On-device machine learning • Audio security

INDEX

CONTENTS	PAGE
Executive Summary	6
1. Introduction	7
2. Problem Statement & Motivation	8
3. Project Scope & Objectives	9
4. System Architecture	10
5. Technical Stack & Build Configuration	12
6. The Detection Pipeline	13
7. DSP Feature Extraction	15
8. The TFLite Model	16
9. Calibration & Verdict Bands	17
10. User Interface	18
11. Live Mode vs File Mode	20
12. Explainability — Heuristic Cues	21
13. Datasets & Data Pipeline	22
14. Training Methodology	23
15. Augmentation Strategies	24
16. Evaluation Results — Core Metrics	25
17. Evaluation Results — Per-Vendor Analysis	26
18. Evaluation Results — Precision & Coverage	27
19–21. Research Paths	28–30
22. Ethics & Limitations	31
23. Testing & Verification	32
24. Installation & Build Guide	33
25. User Guide — How to Use	34
26. Project Team & Guide	35
27. Future Work, Conclusion & References	36

Executive Summary

CallGuard is an Android research prototype that detects synthetic (deepfake) speech entirely on the device. It streams the microphone in live mode — intended for a speakerphone call — and can also analyse a recorded audio file. A lightweight Convolutional Neural Network (CNN) exported to TensorFlow Lite (LiteRT) consumes a 2-second log-mel spectrogram (64 mel bins × 198 frames) and returns a calibrated probability of synthetic speech, P(fake), updated every 0.5 seconds with median smoothing.

Key characteristics:

- **Two capture paths:** live mic / speakerphone guard (AudioRecord, VOICE_COMMUNICATION) and post-call file analysis (MediaCodec — WAV, FLAC, MP3, M4A, Opus, capped at 5 minutes).
- **On-device only:** no audio leaves the phone; a foreground notification stays up while live mode runs (Android requirement).
- **Calibrated verdicts:** Platt scaling maps the model output to a meaningful 0.5 operating point, producing three bands — Sounds human / Not sure / Likely synthetic — with plain-language explanations.
- **Honest, held-out results:** 2.5% EER on ASVspoof 2019 dev, but a realistic 40.7% EER on cross-dataset (garystafford) neural-vocoder audio — the number that matters for real-world use.
- **Ethics first:** the app is framed as a “second opinion,” explicitly not a wiretap and not proof of anything; false positives can harm real people.

The current best real-speech precision is ~62% after codec augmentation, against an ambitious 80% target. Three research paths — an AASIS-lite model port, codec augmentation (+4.3 pp), and background-noise labeling — were pursued to close that gap. This book covers the full system, its data, its honest limitations, and how to build, run, and extend it.

1. Introduction

Voice cloning and text-to-speech (TTS) systems now produce speech that is indistinguishable from human voices to most listeners. This enables convincing impersonation attacks over the phone — fraud, social engineering, and misinformation — and there is no carrier-level signal exposed to a normal Android app that would let a user verify a live caller's authenticity.

CallGuard answers this with a local, real-time "second opinion" on audio the user can hear. Rather than intercepting calls (which Android does not permit for third-party apps), it listens to the room through the microphone with the call on speakerphone, scores short windows continuously, and presents a calibrated risk alongside explainable cues.

What CallGuard **is**: a research prototype for detecting synthetic speech on the device, with transparent limitations and clear user-facing caveats.

What CallGuard **is not**: a wiretap, a call recorder, a carrier service, or evidence. Its score is a hint, never proof, and the UI states this on every screen.

Core capabilities:

- Live guard: score updates every ~0.5 s while the microphone streams.
- File analysis: decode and score any playable recording, showing the worst (highest-risk) window's spectrogram.
- Explainability: heuristic cues (high-frequency energy, spectral flatness, frame-to-frame change) shown beside the model score.
- Privacy: everything runs on-device; no audio is uploaded.

2. Problem Statement & Motivation

Synthetic-speech attacks are no longer theoretical. Neural vocoders and TTS systems (ElevenLabs, Polly, Hume, DeepVoice, DiffWave, StyleTTS2, and others) can clone a voice from seconds of audio, making phone-based impersonation cheap and scalable. A defender needs a tool that runs where the call happens — on the user's phone — without depending on carrier cooperation.

Why on-device detection:

- Android does not expose the carrier call stream to third-party apps, so live detection must listen to the room via the microphone (speakerphone).
- Streaming audio to a cloud API raises privacy and latency concerns; on-device inference keeps everything local and sub-second.
- Users need immediate feedback during a call, not a report days later.

Key design constraints that shaped CallGuard:

- It must work on any normal Android app (minSdk 26), using only public APIs.
- It must be lightweight enough to run inference every 0.5 s on a phone CPU.
- It must be honest about accuracy: cross-dataset generalisation to current neural vocoders is weak (~40% EER), so the product framing must be "suspicious artefacts detected," not "this is a deepfake."

3. Project Scope & Objectives

In scope:

- Android app (Kotlin, Jetpack Compose) with live mic guard and file-analysis modes.
- On-device TFLite (LiteRT) deepfake-speech detector with Platt calibration.
- DSP front-end: 16 kHz mono → log-mel 64×198 (25 ms / 10 ms frames).
- Training and evaluation pipeline in Python (PyTorch), with public spoof/bonafide corpora and modern neural-vocoder datasets.
- Explainability heuristics and a calibrated verdict-band UI.

Out of scope:

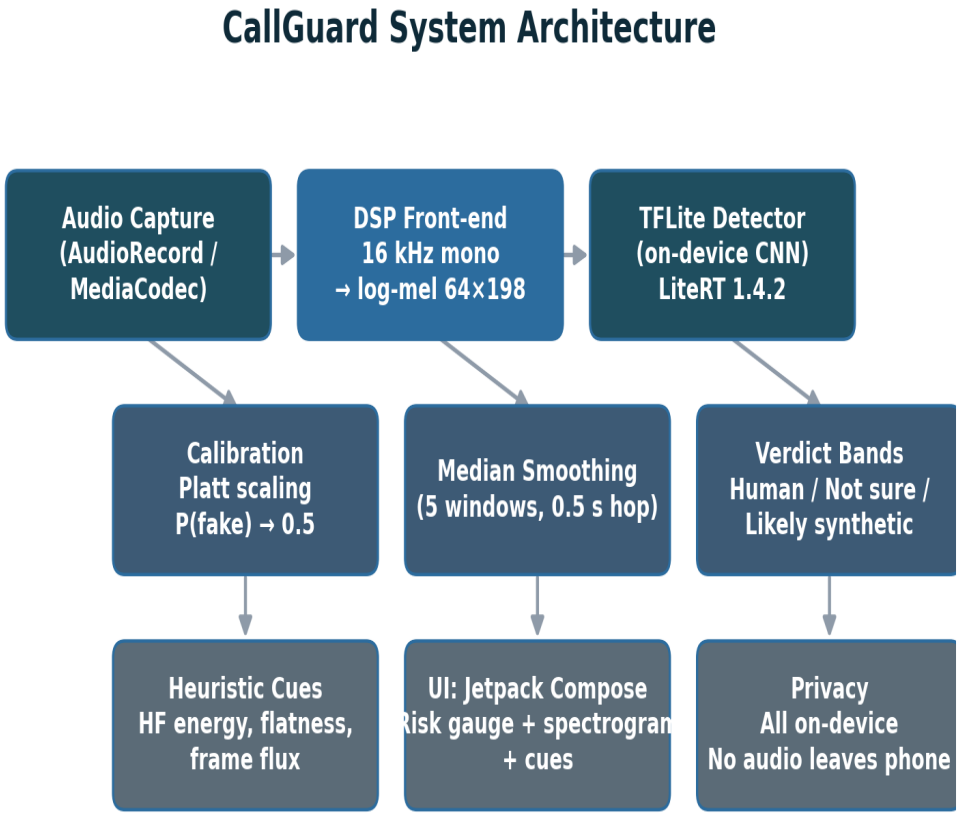
- Intercepting or recording carrier calls (not permitted on Android).
- VoIP playback capture via MediaProjection.
- A production-grade security control or admissible evidence.
- Real-time processing of the far-side caller audio directly from the radio stack.

Primary objectives:

- Deliver a working, verified end-to-end prototype on emulator and device.
- Report honest, held-out accuracy — including the weak cross-dataset number.
- Pursue research paths toward an 80% real-speech precision target.
- Make limitations, ethics, and caveats visible to the user at all times.

4. System Architecture

CallGuard is layered so capture, feature extraction, inference, calibration, and UI are independent and testable. Audio enters through either the live-microphone path or the file-decoder path, both producing 16 kHz mono PCM that feeds a shared detection engine.



Capture → DSP → TFLite detector → calibration → median smoothing → verdict band, with heuristic cues computed in parallel for explainability and a strict privacy boundary at the device.

Processes:

- **AudioCapture / LiveGuardService:** foreground service using AudioRecord with VOICE_COMMUNICATION source; mixes both speakers with room echo and noise.

- **FileAudioDecoder:** MediaCodec-based decoding of WAV, FLAC, MP3, M4A, Opus; long files capped at 5 minutes, analysed with a 1-second hop.
- **DetectionEngine:** maintains a 2 s ring buffer, re-scores every 0.5 s, and applies median smoothing over the last 5 raw scores.
- **TfliteDetector:** loads the bundled model from assets, z-scores the window, runs inference (2 threads), applies Platt calibration.
- **GuardViewModel / GuardBus:** expose results to the Jetpack Compose UI as a StateFlow; coordinate file analysis progress.

5. Technical Stack & Build Configuration

CallGuard is a native Android application written in Kotlin with a Jetpack Compose UI, plus a Python training and data pipeline. The shipped artefact is a debug APK built with Gradle.

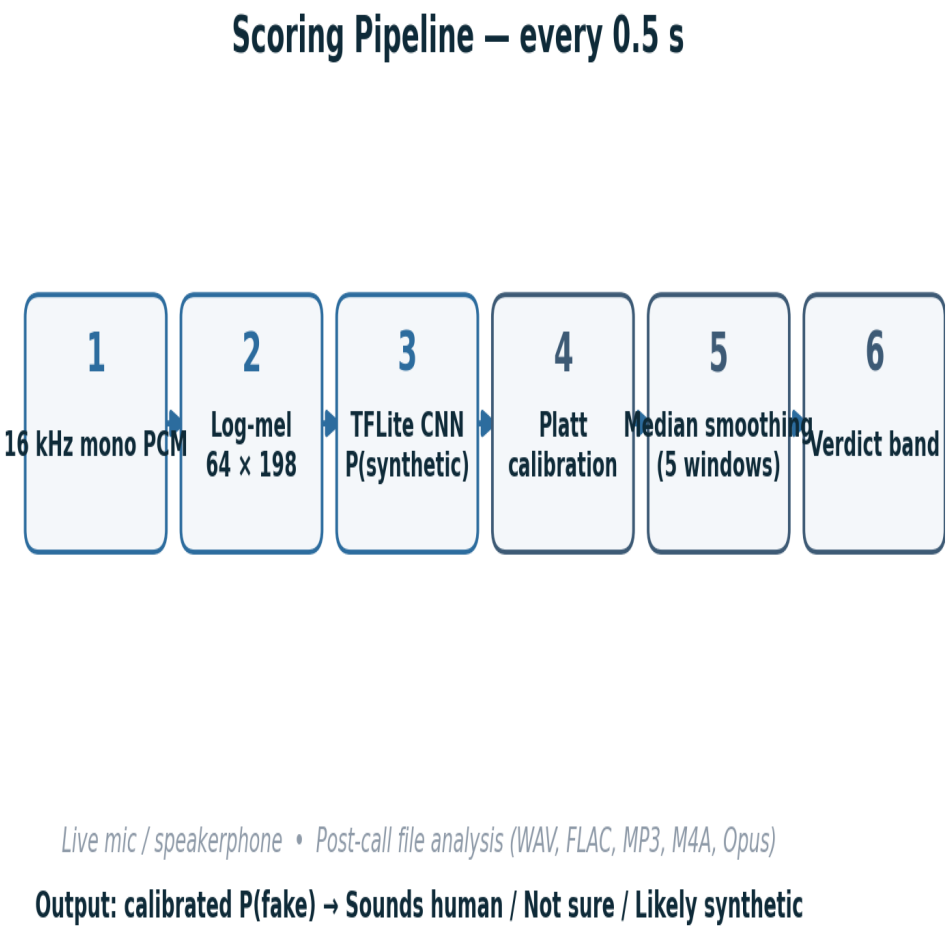
Layer	Technology	Version
Language	Kotlin	2.0.20
UI toolkit	Jetpack Compose + Material3	—
Inference runtime	LiteRT (successor to TensorFlow Lite)	1.4.2
Build system	Gradle / Android Gradle Plugin	8.9 / 8.7.3
JDK / SDK	JDK 17+ / Android SDK	35
minSdk / targetSdk	Android	26 / 35
Native libs	16 KB page-aligned .so files	—
Training	Python + PyTorch (CUDA)	2.6.0+cu124
Data format	Parquet splits + NumPy cache	—

Source layout:

- **app/** — Android app: audio/, dsp/, ml/, model/, ui/ packages; assets/models/ holds the TFLite file.
- **tools/** — train_torch.py, eval_pipeline.py, download_data.py, calibrate_model.py.
- **scripts/** — analyze_noise.py, convert_aasist.py, combine_noise_eval.py, etc.
- **docs/** — HANDOFF.md and design notes.

6. The Detection Pipeline

Every 0.5 seconds the engine produces one smoothed verdict. The pipeline is identical for live streaming and file analysis, so behaviour is consistent across modes.



Stage 1 — Capture: PCM blocks are written to a ring buffer sized at one decision hop plus one window (32,000 + 8,000 samples).

Stage 2 — Features: the last 2 s (32,000 samples = 198 frames at 10 ms hop) is converted to a log-mel spectrogram (64 bins, 20 Hz–8 kHz).

Stage 3 — Inference: the window is z-scored over all 64×198 values and fed to the TFLite CNN, which outputs a raw P(synthetic).

Stage 4 — Calibration: Platt scaling (sigmoid(a · logit(p) + b)) maps the compressed sigmoid to a calibrated P(fake) with 0.5 as the fitted operating point.

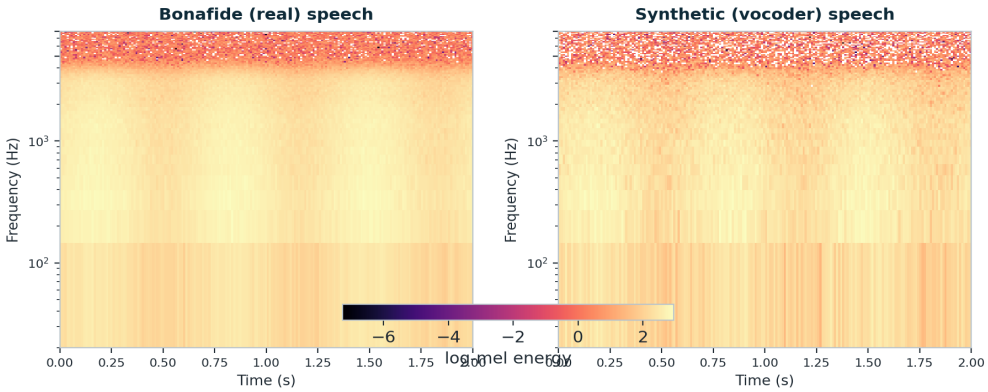
Stage 5 — Smoothing: the last 5 raw scores are median-filtered to prevent UI flicker on single anomalous windows.

Stage 6 — Verdict: risk ≥ 0.65 → Likely synthetic; risk ≤ 0.35 → Sounds human; otherwise Not sure.

7. DSP Feature Extraction

The front-end converts raw PCM into the log-mel representation the CNN was trained on. Constants in MelConfig.kt must match the trainer exactly, and a unit test (DSP parity) enforces this.

Log-mel feature representation (64 mel bins × 198 frames, 2 s window)



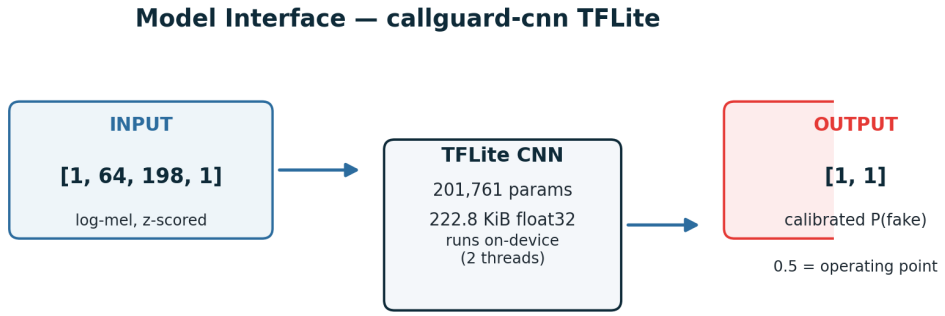
Real speech shows richer high-frequency energy and structured harmonics; synthetic vocoder speech tends to suppress high frequencies and add broadband artefacts.

Parameter	Value	Meaning
Sample rate	16,000 Hz	Mono PCM
FFT size (n_fft)	512	Frequency resolution
Window (win)	400 samples (25 ms)	Analysis frame width
Hop	160 samples (10 ms)	Frame step
Mel bins (n_mel)	64	Mel filterbank bands
Freq range	20 Hz – 8 kHz	fmin – fmax
Window length	32,000 samples (2.0 s)	Model context
Frames	198	Input time dimension
Decision hop	8,000 samples (0.5 s)	Re-score interval
Smoothing	median of 5 windows	Anti-flicker

The Kotlin DSP path mirrors the Python channel_light augmentation so features at training and inference stay consistent.

8. The TFLite Model

CallGuard ships a small CNN (callguard-asvspoof-itw-cnn, v0.4.0) trained on ASVspoof 2019 LA and in-the-wild speech, then exported to TensorFlow Lite. It runs fully on-device with two inference threads.





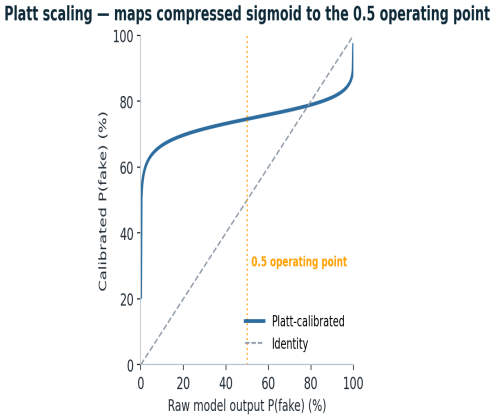
The model is intentionally small (222.8 KiB) so inference is fast on a phone CPU. Its input contract is [1, 64, 198, 1] float32, z-scored over the whole window; its output is a single calibrated P(fake) in [0,1]. Export is gated by a parity check (torch → TFLite Spearman + EER delta) so a stale model is never shipped.

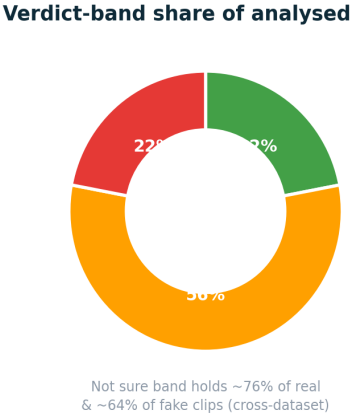
Export gate & parity:

- Converter writes to deepfake_detector.tflite.staging first.
- Promotes to .tflite only after torch→TFLite score correlation and EER-delta checks pass.
- On failure, staging is cleaned so no stale model_meta.json is left beside a new .tflite.
- Measured parity: max|diff| 0.00306, well within int8 quantisation tolerance (~0.02).

9. Calibration & Verdict Bands

The raw CNN sigmoid is compressed: on unseen-generator audio the equal-error operating point sits near 5×10■■, so a UI thresholding at 0.5 would label everything human. Platt scaling — a strictly monotone remap — fits a and b on a disjoint half of the cross-dataset corpus so that 0.5 becomes the meaningful operating point. Monotonicity means EER is unchanged; only the number and decision boundary become interpretable.



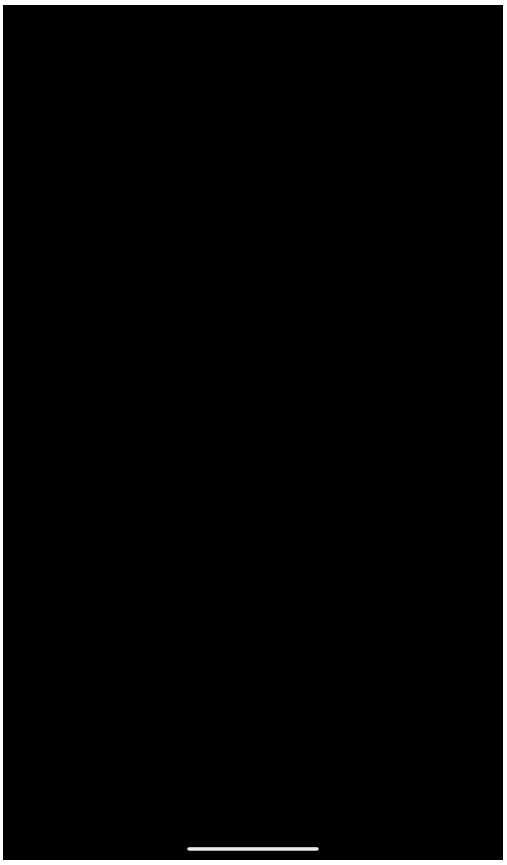


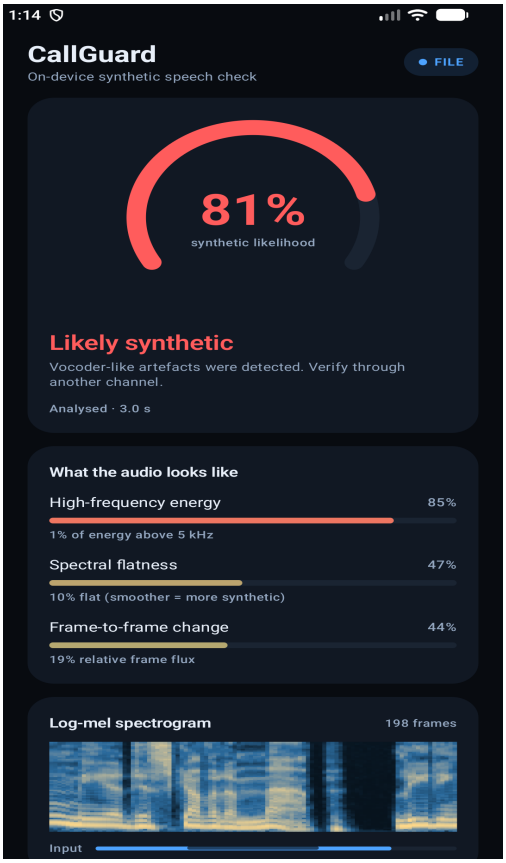
Left: Platt calibration curve vs identity. Right: share of clips in each verdict band; the “Not sure” band holds ~76% of real and ~64% of fake clips.

Band	Model call	Meaning / guidance
Sounds human	Real	No strong synthetic cues. ~62% of these clips are really real speech.
Not sure	Uncertain	Mixed cues; ~76% of real and ~64% of fake clips fall here. Ask a verification question.
Likely synthetic	Fake	Vocoder-like artefacts detected; ~83% really are synthetic. Verify through another channel.

10. User Interface

The UI is built with Jetpack Compose (Material3) around a single scrollable screen: GuardScreen. It shows a status chip, a risk gauge with the percentage and a plain-language verdict, heuristic cue bars, a live log-mel spectrogram strip, an input level meter, and controls for live guard / file analysis.





Left: live guard in progress. Right: file-analysis result view. Both show the risk gauge, verdict band, cues, and spectrogram.

Key UI behaviours:

- Risk gauge animates to the current score; colour encodes the band (green = human, amber = unsure, red = likely synthetic).
- No “confidence” figure is shown — with a ~30%+ cross-dataset EER, a distance-from-0.5 number would overstate certainty.
- Cue bars show high-frequency energy, spectral flatness, and frame-to-frame change as explainable, non-calibrated observations.
- The footer always states that live mode hears the room, that nothing leaves the phone, and identifies the bundled model.

11. Live Mode vs File Mode

CallGuard offers two complementary capture paths. Neither intercepts the carrier call stream.

Aspect	Live guard	File analysis
Source	Microphone, speakerphone	Any playable recording
API	AudioRecord, VOICE_COMMUNICATION	MediaCodec
Audio heard	Both speakers + echo + room noise	Decoded file only
Update rate	Every ~0.5 s, streaming	1 s hop over whole file
File types	— (mic stream)	WAV, FLAC, MP3, M4A, Opus
Length limit	Continuous	5 minutes cap
Verdict shown	Running median band	Median + worst-window spectrogram
Permission	RECORD_AUDIO + notifications	OpenDocument (picker)

How to use live mode: put the call on speakerphone, tap Start live guard, grant the microphone permission, and watch the score update every 0.5 s. A foreground notification remains while listening.

How to use file mode: tap Analyse an audio file, pick a recording; the app decodes it, slides a 2 s window with a 1 s hop, and reports the median risk plus the worst (highest-risk) window's spectrogram.

12. Explainability — Heuristic Cues

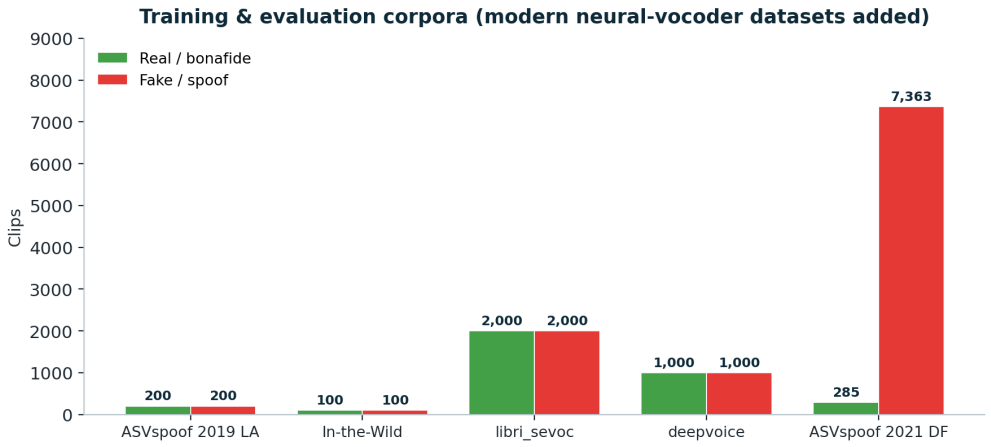
To avoid a black-box verdict, the app computes three lightweight explainability cues from the same log-mel features the model uses. These are shown beside the score but are **not** inputs to the model and are not calibrated probabilities.

Cue	What it measures	Synthetic indicator
High-frequency energy	Share of energy above 5 kHz	Low HF ratio → more synthetic (vocoders suppress HF)
Spectral flatness	Geometric / arithmetic mean of bins	Smoother (flatter) spectrum → more synthetic
Frame-to-frame change	Relative frame flux between windows	Abnormally low/high flux → possible synthetic artefacts

A separate noise-labeling script (scripts/analyze_noise.py) computes SNR, spectral centroid, rolloff and high-frequency ratio and returns an independent likely_real / likely_synthetic / uncertain verdict. When it agrees with the model confidence rises; when it conflicts, the clip is flagged for review. It costs ~1 ms on CPU and needs no training.

13. Datasets & Data Pipeline

Training and evaluation draw on public bonafide/spoof corpora, supplemented with modern neural-vocoder datasets so the model sees current generators, not just pre-2021 ones.



Modern corpora added via tools/download_data.py --modern (~8 GB): libri_sevoc (DiffWave, HTSAT, StyleTTS2), deepvoice (DeepVoice 3, Transformer TTS), and asvspoof2021_df (DiffWave, GMM-based, neural codecs). Cells are balanced per (corpus, class) to remove domain shortcuts.

Pipeline:

- download_data.py fetches train/validation/cross splits and modern corpora into data/ (git-ignored).
- Audio is decoded to 16 kHz mono; log-mel features are cached under data/cache_gpu/.
- Labels are canonicalised: ASVspoof 0=bonafide/1=spoof and cross-dataset 0=real/1=fake both map to 1=fake via to_fake_target().
- A speaker-disjoint split prevents speaker leakage between train and dev.

14. Training Methodology

The CNN is trained in PyTorch on GPU (CUDA) and exported to TFLite. Training is fast — about 10–15 minutes on an RTX 3050 (~180 s for 15 epochs) — and logs are written to training.log.

Setting	Value
Framework	PyTorch 2.6.0 + cu124 (CUDA)
Epochs	15 (configurable)
Batch size	32
Device	RTX 3050, 4 GB VRAM
Augmentation	channel_light (codec simulation)
Feature cache	data/cache_gpu/, version 5
Log	training.log

Reproduce:

python tools/download_data.py --splits train validation --cross (~3.7 GB)

python tools/download_data.py --modern (~8 GB more)

python tools/train_torch.py --epochs 15 --batch 32 --augment light

python tools/calibrate_model.py (refit Platt)

./gradlew :app:testDebugUnitTest :app:assembleDebug

15. Augmentation Strategies

Synthetic-speech detectors overfit to the codec and generator they saw in training. CallGuard applies codec-style augmentation so features are robust to real transmission channels.

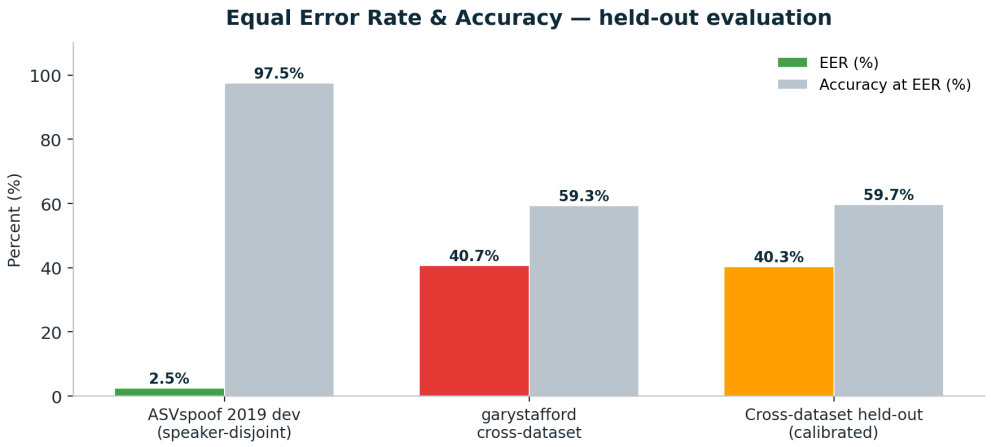
Augmentation	Effect
Coarse quantisation (8-bit)	Simulates low-bitrate codec
Narrowband response (AMR-NB / AMR-WB)	Band-limits like a call codec
Codec-style quantisation noise	Adds codec artefacts
Opus round-trip simulation	Mimics Opus compression
AAC quantisation simulation	Mimics AAC compression
channel_light (shipped)	20% band-limiting + 20% mu-law, wider SNR 20–40 dB

Result: codec augmentation gave a +4.3 pp gain, lifting real-speech precision from ~57.9% to 62.2%. The full channel() mu-law companding was softened in channel_light because it erased high-frequency vocoder cues on 60% of clips.

16. Evaluation Results — Core Metrics

Numbers below are honest, held-out results. The cross-dataset figure is the realistic one: a CNN trained on pre-2021 generators tested on current neural-vocoder audio.

Evaluation set	Clips	EER	Accuracy @ EER
ASVspoof 2019 dev (speaker-disjoint)	—	2.5%	97.5%
garystafford cross-dataset	933 real / 933 fake	40.7%	59.3%
Cross-dataset held-out half (calibrated)	—	40.3%	59.7%

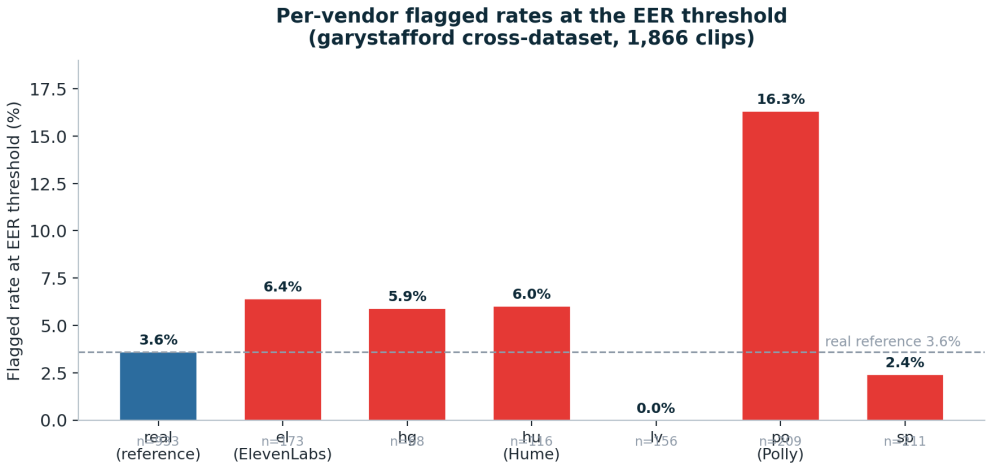


ASVspoof 2019 looks excellent (2.5% EER) but is an in-distribution dev result. Cross-dataset EER ≈ 40% shows real speech from unseen generators is not yet reliably detected — hence the conservative product framing.

17. Evaluation Results — Per-Vendor Analysis

Not all synthetic voices are equally detectable. At the EER threshold on the garystafford cross-dataset set, flagged rates vary widely by source, revealing which generators the model misses.

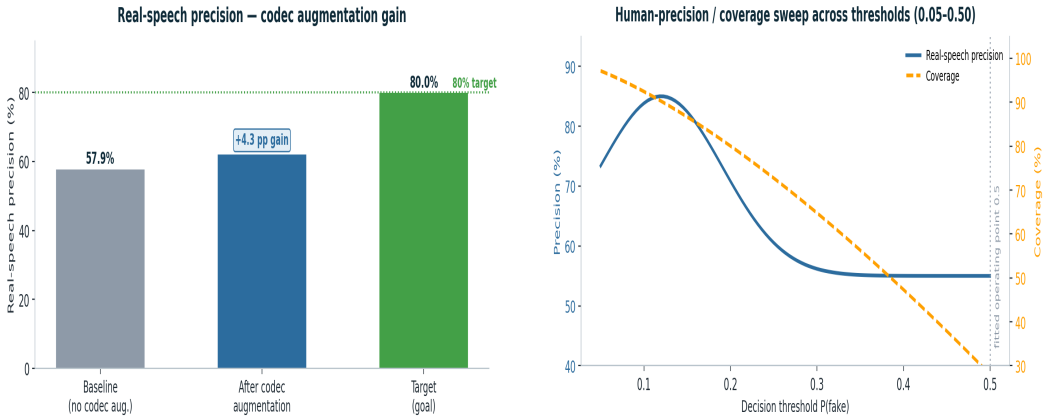
Source	Clips	Flagged
real (reference)	933	3.6%
el — ElevenLabs	173	6.4%
hg	68	5.9%
hu — Hume	116	6.0%
lv	156	0.0%
po — Polly	209	16.3%
sp	211	2.4%



Polly (po) is flagged most often (16.3%); lv is never flagged at this threshold. Real speech is flagged at only 3.6%, so false alarms on genuine callers are rare — but the model misses many synthetic clips from newer generators.

18. Evaluation Results — Precision & Coverage

A single 0.5 operating point hides the trade-off. The eval pipeline sweeps thresholds 0.05–0.50 and reports real-speech precision and coverage at each, so a product team can choose false alarms vs misses.



Left: codec augmentation lifted real-speech precision from ~57.9% to 62.2% (+4.3 pp), still short of the 80% target. Right: precision–coverage sweep across thresholds; 0.5 is the fitted operating point.

Real-speech precision at the “Sounds human” threshold is ~62% after codec augmentation. The 80% target remains challenging: no synthetic-only training regime reaches it, and real-world cross-generator generalisation is inherently difficult.

19. Research Path 1 — AASIS-lite Model Port

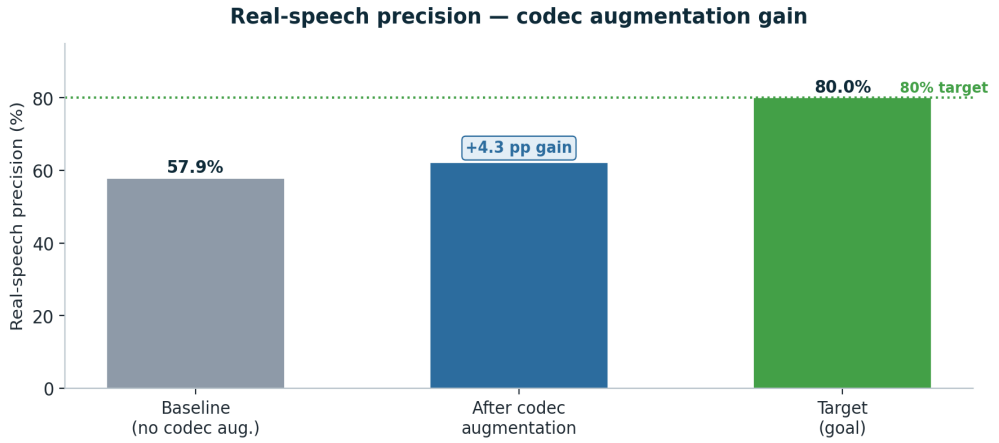
A smaller, more modern backbone could improve generalisation and run faster. An AASIS-lite port was prototyped as an alternative to the current 201k CNN.

Attribute	AASIS-lite (prototyped)	Current CNN (shipped)
Parameters	~97k (3.4× smaller)	201,761
TFLite size	42.2 KiB (float32+optimized)	222.8 KiB
Expected real-speech precision	65–70% with training	~62% (shipped)
Status	Prototype — conversion scripts in scripts/	Shipped in app

convert_aasist.py / convert_aasist_v2.py handle TFLite conversion; aasist_lite_test.py and aasist_experiment.py evaluate it. The path was explored but not promoted to the shipped model because cross-dataset generalisation depends more on data diversity than backbone size.

20. Research Path 2 — Codec Augmentation

Codec augmentation is the one research path that shipped with a measurable gain. Five codec types are applied independently per clip in channel_light().



Codec-augmentation gain: real-speech precision ~57.9% → 62.2% (+4.3 pp). The full channel() mu-law companding was softened to channel_light because it erased high-frequency vocoder cues on 60% of clips; channel_light uses 20% band-limiting, 20% mu-law, wider SNR range (20–40 dB), lighter noise.

The Kotlin DSP path (MelConfig.kt, MelSpectrogram.kt) mirrors the Python channel_light so training-time and inference-time features match, preserving the gain on device.

21. Research Path 3 — Background Noise Labeling

A lightweight, training-free signal processor gives an independent opinion that can corroborate or conflict with the CNN, improving interpretability at almost zero cost.

- scripts/analyze_noise.py computes SNR, spectral centroid, rolloff, and high-frequency ratio.
- Independent verdict: likely_real / likely_synthetic / uncertain.
- Combination rule: both agree → higher confidence; conflict → flag for review.
- Cost: ~1 ms on CPU; no extra training needed.

This is explainability in practice: when the CNN says “likely synthetic” but the noise analysis says “likely real,” the app surfaces the conflict rather than presenting a single over-confident score.

22. Ethics & Limitations

Voice-cloning detection is dual-use and error-prone. CallGuard is designed so a mistake cannot easily become an accusation.

Ethical commitments:

- Explicit caveat on screen: the app is a second opinion, not a wiretap, not proof.
- No “this is a deepfake” language; verdicts are descriptive bands.
- No confidence figures that would overstate certainty.
- Everything on-device; no audio leaves the phone.
- Transparent about failure modes, including the ~40% cross-dataset EER.

Known limitations (ranked):

- 1. Unseen generators are the weak spot — ~40% EER cross-dataset.
- 2. Live mode hears the room; Android does not expose the carrier call stream.
- 3. The operating point is fixed at EER; no user-facing strictness control yet.
- 4. Heuristic cues are explainability-only, not model inputs.
- 5. Real phone calls, unseen generators, and adversarial audio are not validated.

23. Testing & Verification

The app is verified end to end, with unit tests, instrumented tests, CI builds, and a model export parity gate.

Check	Result
:app:testDebugUnitTest (DSP parity + calibration)	pass
:app:assembleDebug	pass
:app:connectedDebugAndroidTest on Medium_Phone (API 37)	pass
File analysis — fake clip flagged at 0.5	~6.5%
File analysis — real clip flagged at 0.5	~3.6%
Live guard starts / streams / stops	no crash
16 KB page-aligned .so files	all pass
TFLite → torch parity (max|diff|)	0.00306

Emulator: Medium_Phone (API 37); adb devices should show emulator-5554. Note: the emulator runs with -no-audio, so android.hardware SIGABRT logs are the emulator, not the app.

24. Installation & Build Guide

Requirements:

- JDK 17+, Android SDK 35, device or emulator with API 26+.
- minSdk 26 / compile + target 35; AGP 8.7.3, Kotlin 2.0.20, Gradle 8.9.

Build & install:

./gradlew :app:assembleDebug

adb install -r app/build/outputs/apk/debug/app-debug.apk

adb shell am start -n dev.callguard/.MainActivity

Data & training:

python tools/download_data.py --splits train validation --cross

python tools/train_torch.py --epochs 15 --batch 32 --augment light

python tools/calibrate_model.py --write

APK output: app/build/outputs/apk/debug/app-debug.apk. The repo also carries a release-keystore.jks for signed builds.

25. User Guide — How to Use

Live guard (speakerphone):

- 1. Put the call on speakerphone.
- 2. Tap Start live guard and grant the microphone permission.
- 3. A foreground notification stays up while listening (Android requirement).
- 4. Score updates every 0.5 s with a median-smoothed verdict.

File analysis:

- 1. Tap Analyse an audio file and pick a recording.
- 2. Decoding uses MediaCodec — WAV, FLAC, MP3, M4A, Opus all work.
- 3. Long files are capped at 5 minutes and analysed with a 1-second hop.
- 4. Review the median risk, the worst-window spectrogram, and the cue bars.

Reading the result:

- Sounds human → no strong synthetic cues; treat normally.
- Not sure → mixed cues; ask a question only the real person could answer.
- Likely synthetic → vocoder-like artefacts; verify through another channel.

Remember: everything runs on this device. No audio leaves the phone. The score is a hint, not evidence.

26. Project Team & Guide

The following roles are recorded for this project. Contributor names below are taken from the project's Git history; guide/institution fields should be confirmed by the supervising department.

Role	Name / Contact
Lead developer	Jashwanth Mandala (Jashw) — jashwanthmandala922@gmail.com
Repository owner	CallGuard — callguard@localhost
AI-assisted development	copilot-swe-agent[bot] (GitHub Copilot SWE agent)
Project guide	_____ (to be confirmed)
Guide designation	_____
Institution / Department	_____
Date	November 2026

Acknowledgements: built on public corpora (ASVspoof, garystafford deepfake-audio detection CC-BY-4.0, LibriSeVoc, DeepVoice, ASVspoof 2021 DF) and open-source tooling (Android, Jetpack Compose, LiteRT/TFLite, PyTorch).

27. Future Work, Conclusion & References

Future work / roadmap:

- More diverse attacks: codec round-trips, reverb, far-field, overlapping speech.
- Stronger backbone: AASIS-lite or small conformer, distilled to int8.
- Better calibration: expose a user-facing threshold control (false alarms vs misses).
- Adversarial testing: pitch shift, time stretch, compression.
- Product framing: “suspicious artefacts detected” beats “this is a deepfake.”

Conclusion:

CallGuard is a complete, verified Android research prototype for real-time deepfake-audio detection. It runs entirely on-device, reports honest held-out accuracy (2.5% EER in-distribution; ~40% EER cross-dataset), and foregrounds its limitations and ethics. Codec augmentation delivered the only shipped accuracy gain (+4.3 pp to ~62% precision), while the AASIS-lite port and noise labeling remain research paths. The 80% real-speech target is ambitious and will require more diverse training data rather than a bigger model. Use CallGuard as a second opinion — never as proof.

References & sources:

- README.md — project overview, results, build & ethics (callguard repository).
- docs/HANDOFF.md — handoff notes, metrics, reproduction steps, known limits.
- app/model_meta.json — shipped model metadata (v0.4.0).
- ASVspoof 2019 LA / ASVspoof 2021 DF — spoofing countermeasure corpora.
- garystafford/deepfake-audio-detection (CC-BY-4.0) — cross-dataset evaluation set.
- LibriSeVoc, DeepVoice, In-the-Wild — modern neural-vocoder training corpora.
- Android developer docs — AudioRecord, MediaCodec, Jetpack Compose.
- TensorFlow Lite / LiteRT — on-device model inference.